

ĐẠI HỌC BÁCH KHOA HÀ NỘI
Trường Công nghệ Thông tin và Truyền thông

BÁO CÁO ĐỒ ÁN

**Hệ thống định danh người dùng qua hành vi sử dụng
điện thoại**

Môn học: Nhập môn Khoa học dữ liệu
Hướng tiếp cận: Random Forest và CNN 1D
Dữ liệu: Hành vi cảm biến điện thoại và gõ phím

Thành viên nhóm

Lương Mạnh Tường	20225949
Bùi Minh Tuấn	20225678
Trương Quốc Triệu	20225941
Nguyễn Đức Thành	20225930
Nguyễn Anh Tuấn	20225772
Vũ Ngọc Lâm	20225645
Vũ Nhật Quang	20225761
Mai Văn Đăng	20225699
Nguyễn Đình Thành	20225670

Mục lục

Tóm tắt	1
1 Mô tả bài toán thực tế	2
1.1 Bối cảnh	2
1.2 Phát biểu bài toán	2
1.3 Ý nghĩa thực tế	2
2 Phương pháp học máy và dataset sử dụng	2
2.1 Dataset	2
2.2 Tiền xử lý dữ liệu cảm biến	3
2.3 Cách chia dữ liệu	4
2.4 Mô hình Random Forest	5
2.5 Mô hình CNN 1D	5
2.6 Đánh giá open-set	6
3 Kết quả thí nghiệm đánh giá hiệu năng	6
3.1 Chỉ số đánh giá	6
3.2 Kết quả tổng thể	7
3.3 Độ ổn định theo fold	8
3.4 Accuracy của Random Forest	9
3.5 Phân tích CNN theo hoạt động	9
3.6 Kết quả open-set	10
4 Các chức năng chính của hệ thống và cách sử dụng	10
4.1 Chức năng chính	10
4.2 Cài đặt môi trường	11
4.3 Chạy pipeline đơn giản	11
4.4 Chạy bằng JupyterLab	12
5 Cấu trúc mã nguồn và vai trò các class/method chính	12
5.1 Tổ chức mã nguồn	12
5.2 Vai trò các module chính	12
5.3 Class và method quan trọng	13
6 Khó khăn gặp phải và cách giải quyết	13

6.1	Tên file và schema dữ liệu không đồng nhất	13
6.2	Nguy cơ rò rỉ dữ liệu khi chia train/validation	13
6.3	Ít file ở một số người dùng	13
6.4	Mất cân bằng số cửa sổ giữa các file	14
6.5	Dữ liệu cảm biến có nhiều và khác tần số	14
6.6	CNN không vượt Random Forest	14
6.7	Open-set còn yếu	14
6.8	Tái lập thí nghiệm open-set	14
7	Khám phá mới hoặc kết luận	14
7.1	Khám phá chính	14
7.2	Kết luận	15
7.3	Hướng phát triển	15
7.4	Đóng gói sản phẩm nộp	15

Tóm tắt

Đồ án khảo sát bài toán định danh người dùng từ dữ liệu sử dụng điện thoại. Phần mô hình chính dùng tín hiệu cảm biến quán tính; log gõ phím được giữ như nguồn dữ liệu phụ để kiểm tra và mở rộng sau. Hai hướng được so sánh là Random Forest trên đặc trưng thủ công và CNN 1D trên chuỗi cảm biến. Với cách đánh giá 3-fold *StratifiedGroupKFold*, Random Forest đạt macro-F1 trung bình 0.7845, cao hơn CNN 1D simple đạt 0.6881. Kết quả này cho thấy trong tập dữ liệu còn ít phiên ghi, đặc trưng thống kê được thiết kế thủ công vẫn là baseline mạnh và dễ giải thích.

1 Mô tả bài toán thực tế

1.1 Bối cảnh

Điện thoại cá nhân thường chứa tài khoản, tin nhắn, ảnh và nhiều dữ liệu nhạy cảm. Mật khẩu, vân tay hoặc nhận diện khuôn mặt bảo vệ tốt ở thời điểm mở khoá, nhưng không theo dõi liên tục trong quá trình sử dụng. Sinh trắc học hành vi tiếp cận vấn đề từ một góc khác: nhận ra người dùng qua cách họ cầm, di chuyển và thao tác với thiết bị.

Trong đề án này, hệ thống cần dự đoán người đang sử dụng điện thoại từ dữ liệu hành vi. Nguồn chính là chuyển động của điện thoại khi người dùng ngồi, đứng và đi bộ; dữ liệu gõ phím được dùng để ghi nhận thêm nhịp tương tác.

1.2 Phát biểu bài toán

Với mỗi mẫu dữ liệu cảm biến quán tính có dạng chuỗi thời gian:

$$X_i \in \mathbb{R}^{128 \times 6}$$

trong đó 128 là số mẫu trong một cửa sổ 2.56 giây, 6 kênh gồm `acc_x`, `acc_y`, `acc_z`, `gyro_x`, `gyro_y`, `gyro_z`. Mục tiêu là học hàm phân loại:

$$f(X_i) \rightarrow y_i$$

với y_i là mã người dùng trong tập người dùng đã biết.

Bài toán có hai chế độ đánh giá:

- **Closed-set**: tất cả mẫu kiểm thử thuộc 12 người dùng đã xuất hiện trong huấn luyện.
- **Open-set**: một số người dùng chưa từng xuất hiện trong huấn luyện, hệ thống cần phát hiện tín hiệu “người dùng chưa biết”.

1.3 Ý nghĩa thực tế

Nếu đạt độ tin cậy đủ cao, hướng này có thể dùng cho xác thực liên tục hoặc cảnh báo khi thiết bị có dấu hiệu bị người khác sử dụng. Trong đề án, trọng tâm là xây dựng pipeline dữ liệu có thể chạy lại, chia train/validation đúng cách và giải thích rõ giới hạn của kết quả.

2 Phương pháp học máy và dataset sử dụng

2.1 Dataset

Bộ dữ liệu dùng trong báo cáo gồm 15 người dùng và 185 file CSV sau khi bỏ các log không dùng trong pipeline cuối. Các file còn lại được chia thành hai nhóm:

- **Inertial**: dữ liệu cảm biến quán tính, gồm gia tốc kế và con quay hồi chuyển. Đây là nguồn dữ liệu chính dùng cho mô hình.
- **Keystroke**: dữ liệu gõ phím như thời điểm gõ, khoảng cách thời gian giữa các phím, tốc độ gõ.

Sau bước quét dữ liệu, `manifest.csv` được sinh ra để chuẩn hoá thông tin: người dùng, tên file, modality, hoạt động, session, attempt/rep, số dòng và tần số lấy mẫu ước lượng. Bảng 1 và 2 tóm tắt quy mô dữ liệu sau khi chuẩn hoá manifest.

Bảng 1: Số lượng file theo modality

Modality	Số file
Inertial	154
Keystroke	31
Tổng	185

Bảng 2: Số file inertial theo hoạt động

Hoạt động	Số file inertial
sitting	46
standing	46
walking	62

Mô hình chính dùng inertial vì đây là chuỗi thời gian liên tục và khớp trực tiếp với cả hai hướng mô hình: Random Forest trên đặc trưng thống kê và CNN 1D trên tín hiệu thô đã xử lý. Keystroke được trích đặc trưng trong `features.py` nhưng chưa đưa vào kết quả chính, để phần đánh giá tập trung vào một nguồn tín hiệu rõ ràng. Tần số lấy mẫu thực tế giữa các file không hoàn toàn đồng nhất, vì vậy pipeline có bước resample về 50 Hz trước khi chia cửa sổ.

2.2 Tiền xử lý dữ liệu cảm biến

Pipeline tiền xử lý được cài đặt trong `preprocess.py`. Các bước chính:

1. Đọc dữ liệu cảm biến từ CSV.
2. Điền giá trị thiếu bằng forward-fill/backward-fill.
3. Resample về 50 Hz nếu tần số lấy mẫu thực tế lệch nhiều.
4. Lọc low-pass Butterworth cutoff 20 Hz để giảm nhiễu cao tần.
5. Lọc high-pass Butterworth cutoff 0.3 Hz cho ba trục gia tốc để tách thành phần trọng lực.

6. Chia cửa sổ 128 mẫu, bước trượt 64 mẫu, tương đương overlap 50%.
7. Giới hạn tối đa 300 cửa sổ mỗi file để tránh một file dài chi phối toàn bộ tập huấn luyện.

Kết quả lưu tại `windows.npz` với tensor:

$$X \in \mathbb{R}^{39018 \times 128 \times 6}$$

2.3 Cách chia dữ liệu

Sau tiền xử lý, tập inertial có 39018 cửa sổ, mỗi cửa sổ có kích thước $(128, 6)$. 15 người dùng trong dữ liệu gồm `userA`, `userB`, `userC`, `userD`, `userE`, `userF`, `userG`, `userH`, `userI`, `userK`, `userL`, `userM`, `userN`, `userP`, `userQ`. Nhóm chia dữ liệu theo hai tầng: tầng người dùng cho open-set và tầng file cho cross-validation closed-set.

Bảng 3: Tách known/unknown ở mức người dùng

Tập	Users	Windows	Groups/files
Known closed-set	12	29725	120
Unknown open-set	3	9293	34
Tổng	15	39018	154

Ba người dùng `userD`, `userE`, `userI` được giữ riêng cho bài toán open-set. Đây là chia tách ở mức user, không phải ở mức cửa sổ. Lý do là open-set cần mô phỏng đúng tình huống thực tế: hệ thống gặp một người dùng chưa từng xuất hiện trong giai đoạn huấn luyện. Nếu bất kỳ cửa sổ nào của các user unknown lọt vào train, bài toán sẽ không còn là open-set đúng nghĩa.

Với 12 user known còn lại, nhóm dùng *StratifiedGroupKFold*. Điểm quan trọng là **không chia ngẫu nhiên theo cửa sổ**. Vì sliding window tạo ra nhiều cửa sổ liên tiếp, các cửa sổ gần nhau trong cùng một file có nội dung tín hiệu rất giống nhau. Nếu random split theo cửa sổ, train và validation có thể cùng chứa các đoạn gần như trùng từ một phiên ghi, làm mô hình học đặc trưng của phiên ghi thay vì học hành vi người dùng. Đây là dạng data leakage và thường làm điểm đánh giá cao giả tạo.

Để tránh leakage, mỗi cửa sổ được gán group có dạng `user__file`. Khi chia fold, toàn bộ cửa sổ sinh ra từ cùng một file luôn nằm trọn trong train hoặc trọn trong validation. Như vậy validation mô phỏng một phiên ghi mới hơn, khó hơn và gần thực tế hơn so với random split theo cửa sổ. Thành phần *stratified* giúp phân phối user trong các fold tương đối cân bằng, còn *group* đảm bảo rằng buộc không tách cùng một file sang hai tập. Tham số `shuffle=True`, `random_state=42` được dùng để kết quả có thể tái lập.

Nhóm chọn 3-fold vì số file giữa các user không đều; user ít file nhất là `userQ`, chỉ có 3 file inertial. Nếu dùng 5-fold, một số fold validation có nguy cơ thiếu class, khiến macro-F1 không còn phản ánh đủ 12 user known. Với 3-fold, mỗi fold vẫn có đủ lớp để đánh giá ổn định hơn.

Bảng 4: Thống kê train/validation của 3 fold closed-set

Fold	Train windows	Val windows	Train groups	Val groups
1	20069	9656	80	40
2	19814	9911	82	38
3	19567	10158	78	42

2.4 Mô hình Random Forest

Random Forest là phương pháp để phù hợp với dữ liệu vừa/nhỏ. Mỗi cửa sổ cảm biến được chuyển thành vector 65 đặc trưng thủ công. Cấu trúc feature gồm $6 \times 9 = 54$ đặc trưng thống kê/FFT cho 6 kênh, 9 đặc trưng correlation giữa các cặp trục được chọn, và 2 đặc trưng Signal Magnitude Area (SMA) cho accelerometer và gyroscope.

- Thống kê theo từng trục: mean, standard deviation, MAD, min, max, IQR, energy.
- Đặc trưng miền tần số: FFT entropy, dominant frequency.
- Tương quan giữa các trục.
- Signal Magnitude Area cho accelerometer và gyroscope.

Mô hình sử dụng `RandomForestClassifier` với 300 cây. Tham số cân bằng lớp là `class_weight=balanced_subsample`. Random Forest phù hợp với bài toán này vì các đặc trưng thống kê và tần số tương đối ổn định khi số phiên ghi còn ít, mô hình ít tham số và không cần quá nhiều tinh chỉnh để đạt baseline tốt.

Để tránh leakage ở bước chuẩn hoá, `StandardScaler` chỉ được fit trên train fold. Sau đó scaler này mới được dùng để transform validation fold. Quy trình này đảm bảo validation không tham gia vào việc ước lượng trung bình và độ lệch chuẩn của đặc trưng.

2.5 Mô hình CNN 1D

CNN 1D được dùng để kiểm tra khả năng học trực tiếp từ chuỗi cảm biến. Sau khi đi qua `InertialDataset`, dữ liệu đầu vào có dạng $(B, 6, 128)$, trong đó B là batch size, 6 là số kênh cảm biến, 128 là chiều thời gian. Conv1D trượt kernel theo trục thời gian để học các mẫu biến thiên ngắn hạn của accelerometer và gyroscope.

```
Input: (6, 128)
Conv1D(6 -> 32, kernel=7) + BatchNorm + ReLU + MaxPool
Conv1D(32 -> 64, kernel=5) + BatchNorm + ReLU + MaxPool
Conv1D(64 -> 128, kernel=3) + BatchNorm + ReLU
AdaptiveAvgPool1D
Dropout(0.3)
Linear(128 -> 12)
```


Trong kiến trúc này, BatchNorm giúp ổn định phân phối activation trong quá trình huấn luyện; ReLU tạo phi tuyến; MaxPool giảm chiều thời gian và giữ mẫu nổi bật; AdaptiveAvgPool gom toàn bộ chuỗi thành một vector cố định trước khi phân loại; Dropout giảm overfit. Kiến trúc được giữ ở mức 3 tầng convolution để báo cáo dễ giải thích và không biến phần mô hình thành một sweep phức tạp.

Cấu hình huấn luyện:

- Optimizer: Adam.
- Learning rate: 0.001.
- Loss: CrossEntropy có class weight nghịch tần suất.
- Epoch: 40.
- Batch size: 128.
- Không dùng scheduler, không dùng MixUp/CutMix, không dùng sweep nhiều phase.

Việc không dùng scheduler hoặc augmentation phức tạp là lựa chọn có chủ đích cho bản báo cáo cuối: kết quả có thể thấp hơn một số thử nghiệm nâng cao, nhưng dễ tái lập, dễ trình bày và phù hợp hơn với đồ án nhập môn khoa học dữ liệu.

2.6 Đánh giá open-set

Với người dùng chưa biết, nhóm dùng hai cách tính điểm:

- **Softmax max-prob**: nếu xác suất lớn nhất thấp, mẫu có khả năng thuộc unknown.
- **Mahalanobis**: đo khoảng cách embedding tới phân phối Gaussian của từng class đã biết.

Chỉ số dùng cho open-set là AUROC và TPR tại $FPR = 5\%$.

3 Kết quả thí nghiệm đánh giá hiệu năng

3.1 Chỉ số đánh giá

Nhóm sử dụng nhiều chỉ số vì mỗi chỉ số phản ánh một khía cạnh khác nhau của hệ thống:

- **Accuracy**: tỷ lệ dự đoán đúng trên toàn bộ mẫu. Chỉ số này dễ hiểu, nhưng có thể bị ảnh hưởng khi số cửa sổ giữa các user không đều.
- **Macro-F1**: tính F1 riêng cho từng user rồi lấy trung bình. Chỉ số này phù hợp hơn cho bài toán định danh nhiều người dùng vì mỗi user có trọng số như nhau trong điểm cuối.

- **AUROC**: dùng cho open-set, đánh giá khả năng phân biệt known và unknown khi thay đổi ngưỡng quyết định.
- **TPR@FPR=5%**: mô phỏng yêu cầu bảo mật, chỉ cho phép tỷ lệ nhận nhầm unknown thành known ở mức thấp.

Với C class người dùng, macro-F1 được tính theo:

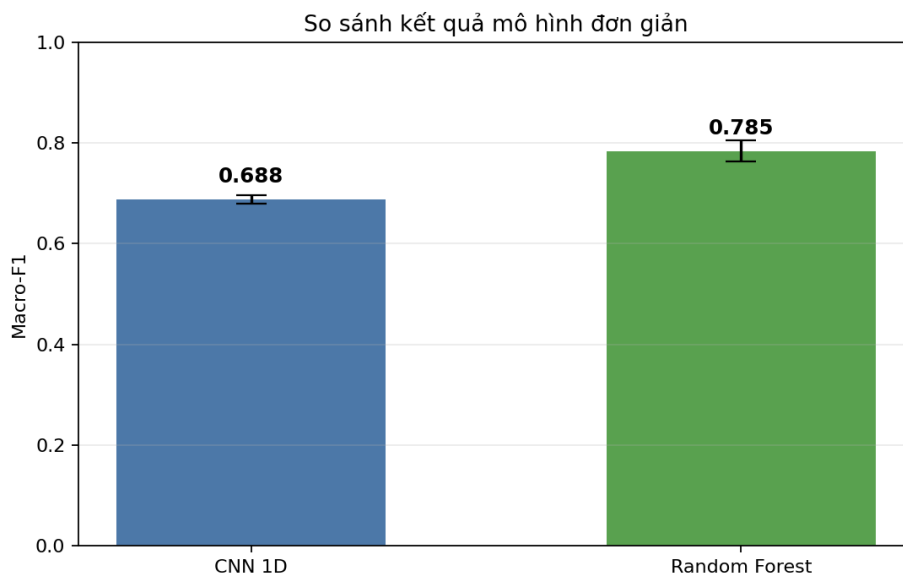
$$MacroF1 = \frac{1}{C} \sum_{c=1}^C F1_c$$

3.2 Kết quả tổng thể

Kết quả được chạy lại với cấu hình đơn giản, lưu tại `simple_results.json`. Bảng 5 trình bày điểm trung bình trên 3 fold.

Bảng 5: Kết quả closed-set theo 3-fold StratifiedGroupKFold

Mô hình	Macro-F1 trung bình	Độ lệch chuẩn	Accuracy trung bình
CNN 1D simple	0.6881	0.0077	—
Random Forest	0.7845	0.0204	0.8013



Hình 1: So sánh macro-F1 giữa CNN 1D và Random Forest

Random Forest đạt 0.7845 ± 0.0204 macro-F1 và accuracy trung bình 0.8013. CNN 1D simple đạt 0.6881 ± 0.0077 macro-F1. Như vậy RF cao hơn CNN khoảng 0.0964 macro-F1. Kết quả này không có nghĩa là CNN là mô hình kém, mà cho thấy dữ liệu hiện tại chưa đủ lớn và chưa đủ đa phiên để CNN phát huy hết khả năng học đặc trưng tự động. Trong

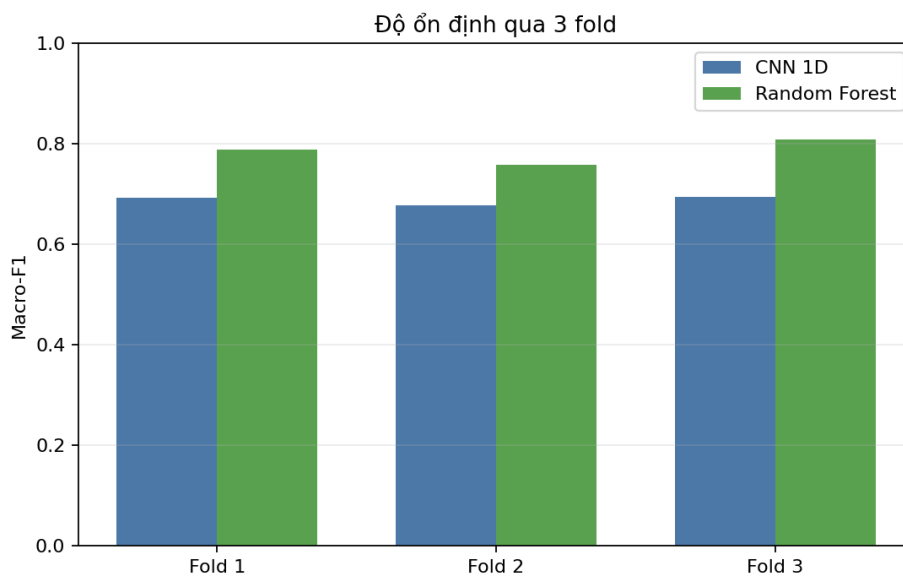
khi đó, feature thống kê thủ công của RF đã mã hoá trực tiếp nhiều thông tin hữu ích về biên độ, năng lượng, tần số và tương quan giữa các trục.

Đây là một kết luận quan trọng của đề án: với dữ liệu chưa lớn, số phiên ghi của mỗi người chưa nhiều, đặc trưng thống kê thủ công kết hợp với mô hình cây vẫn rất cạnh tranh. Vì vậy báo cáo chọn Random Forest là mô hình chính và CNN 1D là mô hình học sâu bổ sung.

3.3 Độ ổn định theo fold

Bảng 6: Macro-F1 trên từng fold

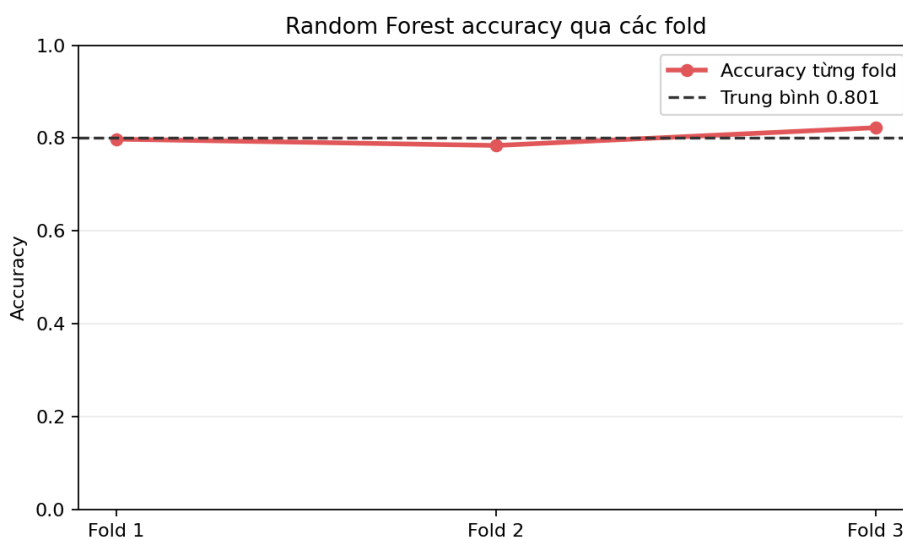
Mô hình	Fold 1	Fold 2	Fold 3
CNN 1D simple	0.6933	0.6772	0.6938
Random Forest	0.7876	0.7582	0.8077



Hình 2: Macro-F1 của từng mô hình qua 3 fold

CNN có độ lệch chuẩn nhỏ hơn, tức kết quả giữa các fold ổn định hơn, nhưng mức điểm trung bình thấp hơn. Random Forest dao động nhiều hơn một chút giữa các fold nhưng vẫn là mô hình có hiệu năng tốt nhất. Điều này phù hợp với trực giác: RF tận dụng feature thủ công tốt hơn trong dữ liệu nhỏ, còn CNN cần thêm dữ liệu hoặc thêm phiên ghi để học biểu diễn bền vững hơn.

3.4 Accuracy của Random Forest

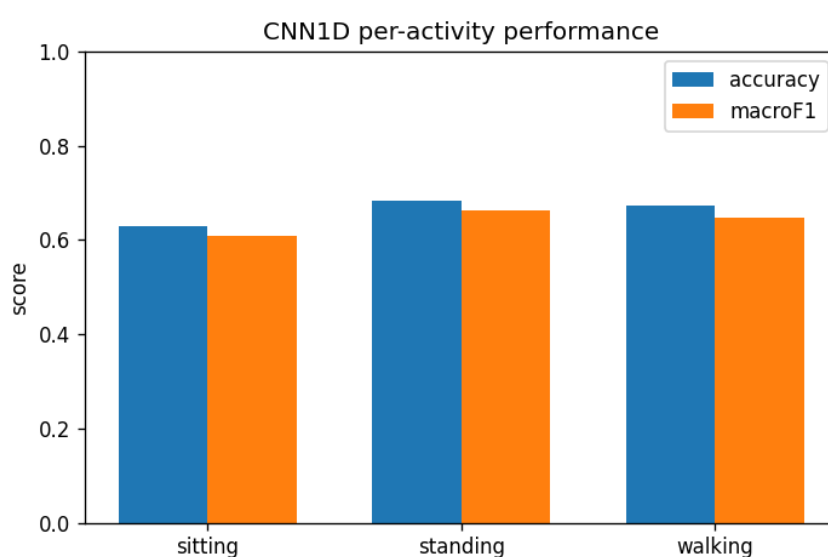


Hình 3: Accuracy của Random Forest qua từng fold

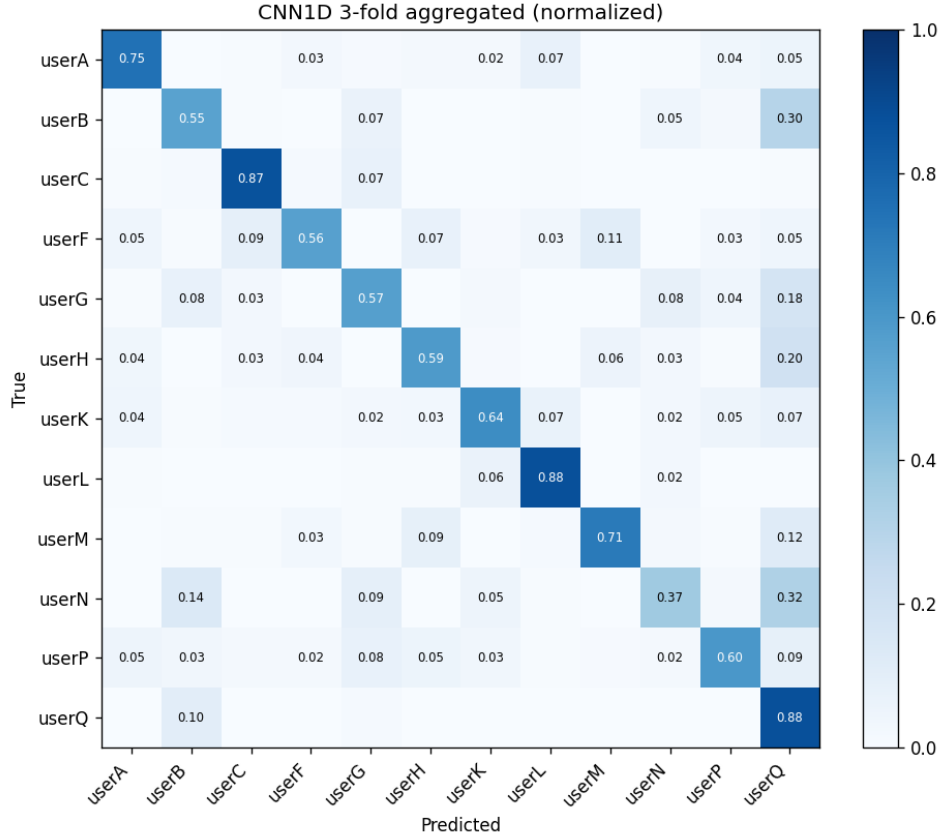
Accuracy trung bình của Random Forest đạt 0.8013. Chỉ số này dễ hiểu với người nghe không chuyên, còn macro-F1 được ưu tiên trong đánh giá chính vì dữ liệu giữa các người dùng không hoàn toàn cân bằng.

3.5 Phân tích CNN theo hoạt động

Kết quả cũ của pipeline đánh giá tổng hợp cũng tạo được biểu đồ theo hoạt động và ma trận nhầm lẫn. Hai hình này dùng để phân tích sâu hơn các lỗi của mô hình CNN.



Hình 4: Hiệu năng CNN theo hoạt động sitting, standing, walking



Hình 5: Ma trận nhầm lẫn chuẩn hoá của CNN

3.6 Kết quả open-set

Bảng 7: Kết quả phát hiện người dùng chưa biết

Phương pháp	AUROC trung bình	TPR @ FPR = 5%
Softmax max-prob	0.6702	0.2782
Mahalanobis	0.6124	0.0807

Open-set chưa phải điểm mạnh của hệ thống hiện tại. Softmax max-prob đạt AUROC 0.6702, tốt hơn Mahalanobis nhưng vẫn chỉ nên xem là baseline. AUROC khoảng 0.67 cho thấy có tín hiệu phân biệt known/unknown, nhưng chưa đủ tốt để triển khai thực tế. Nguyên nhân chính là embedding học từ dữ liệu nhỏ chưa tạo biên phân tách đủ ổn định, đặc biệt khi số phiên ghi mỗi user còn hạn chế.

4 Các chức năng chính của hệ thống và cách sử dụng

4.1 Chức năng chính

Hệ thống được tổ chức thành các bước độc lập, có thể chạy lại từng phần.

Bảng 8: Các chức năng chính của hệ thống

STT	Lệnh	Chức năng
1	<code>python-msrc.io</code>	Quét dữ liệu thô, parse tên file, sinh <code>manifest.csv</code> .
2	<code>python-msrc.preprocess</code>	Tiền xử lý cảm biến, lọc tín hiệu, chia cửa sổ, sinh <code>windows.npz</code> .
3	<code>python-msrc.features</code>	Trích đặc trưng gõ phím phục vụ phân tích bổ sung.
4	<code>python-mscripts.run_simple_pipeline</code>	Chạy pipeline đơn giản gồm CNN 1D và RF, lưu <code>simple_results.json</code> .
5	<code>python-mscripts.plot_simple_results</code>	Vẽ biểu đồ so sánh mô hình, fold và accuracy.
6	<code>python-msrc.openset</code>	Đánh giá phát hiện user unknown bằng softmax/Mahalanobis.
7	<code>jupyterlabsimple_pipeline.ipynb</code>	Chạy notebook minh họa kết quả và biểu đồ trong JupyterLab.

4.2 Cài đặt môi trường

Yêu cầu Python 3.10 trở lên. Các gói chính được liệt kê trong `requirements.txt`: `numpy`, `pandas`, `scipy`, `scikit-learn`, `torch`, `matplotlib`, `seaborn`, `joblib`, `tqdm`, `jupyterlab`.

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt --index-url https://pypi.org/simple
```

4.3 Chạy pipeline đơn giản

```
source venv/bin/activate
python -m scripts.run_simple_pipeline
python -m scripts.plot_simple_results
```

Kết quả chính:

- `simple_results.json`: điểm số CNN và RF.
- `simple_model_comparison.png`: so sánh tổng thể.
- `simple_fold_macro_f1.png`: điểm từng fold.
- `simple_rf_accuracy.png`: accuracy của RF.

4.4 Chạy bằng JupyterLab

Nhóm đã chuẩn bị notebook `simple_pipeline.ipynb`. Mở notebook bằng:

```
source venv/bin/activate
jupyter lab simple_pipeline.ipynb
```

Trong notebook, biến `RUN_TRAIN` mặc định là `False` để không train lại CNN mỗi lần mở. Nếu muốn chạy lại toàn bộ, đổi `RUN_TRAIN = True`. Nếu chỉ muốn cập nhật RF và biểu đồ, đổi `RUN_RF_ONLY = True`.

5 Cấu trúc mã nguồn và vai trò các class/method chính

5.1 Tổ chức mã nguồn

Mã nguồn được tách theo đúng luồng xử lý: đọc dữ liệu, tiền xử lý, định nghĩa dataset, huấn luyện, đánh giá và vẽ hình. Trong báo cáo chỉ nêu tên file chính để tránh làm phần trình bày bị nặng bởi đường dẫn thư mục.

5.2 Vai trò các module chính

Bảng 9: Vai trò các module chính

File	Vai trò
<code>config.py</code>	Khai báo đường dẫn, kích thước cửa sổ, danh sách kênh cảm biến, user unknown, seed.
<code>io.py</code>	Parse tên file, đọc CSV, ước lượng tần số lấy mẫu, xây dựng manifest.
<code>preprocess.py</code>	Resample, lọc Butterworth, tách trọng lực, chia sliding window.
<code>features.py</code>	Trích đặc trưng thống kê, FFT, correlation, SMA và đặc trưng keystroke.
<code>datasets.py</code>	Định nghĩa <code>InertialDataset</code> , chuẩn hoá dữ liệu, chuyển shape cho Conv1D.
<code>models.py</code>	Định nghĩa CNN1D và factory <code>make_model</code> .
<code>train.py</code>	Chạy huấn luyện 3-fold, lưu checkpoint và log experiment.
<code>evaluate.py</code>	Chạy Random Forest baseline, gom prediction CNN, vẽ confusion/per-activity.
<code>openset.py</code>	Đánh giá softmax max-prob và Mahalanobis cho open-set.

File	Vai trò
<code>run_simple_pipeline.py</code>	Script đơn giản hoá để chạy đúng hai mô hình chính: CNN 1D và RF.
<code>plot_simple_results.py</code>	Tạo ba biểu đồ chính dùng trong báo cáo.

5.3 Class và method quan trọng

- `InertialDataset.__getitem__`: lấy một cửa sổ cảm biến, chuẩn hoá theo mean/std của train fold, chuyển từ shape (T, C) sang (C, T) cho PyTorch Conv1D.
- `CNN1D.forward`: chạy chuỗi Conv1D, BatchNorm, ReLU, pooling và linear head để dự đoán người dùng.
- `CNN1D.embed`: trả embedding trước lớp phân loại, dùng cho open-set.
- `run_cv`: thực hiện vòng lặp 3-fold, gọi train từng fold, lưu checkpoint và metric.
- `rf_baseline`: trích đặc trưng thủ công, chuẩn hoá, train Random Forest và tính macro-F1/accuracy.
- `plot_model_comparison`, `plot_fold_scores`, `plot_rf_accuracy`: sinh hình minh hoạ kết quả.

6 Khó khăn gặp phải và cách giải quyết

6.1 Tên file và schema dữ liệu không đồng nhất

Các file dữ liệu có nhiều quy ước đặt tên khác nhau, ví dụ `_s1_att2.csv`, `_s1_r3.csv`, hoặc thiếu một số thành phần. Nếu parse thủ công từng trường hợp, pipeline dễ lỗi và khó mở rộng. Nhóm xử lý bằng các regex riêng trong `io.py`, sau đó sinh manifest thống nhất.

6.2 Nguy cơ rò rỉ dữ liệu khi chia train/validation

Nếu chia ngẫu nhiên theo cửa sổ, các cửa sổ từ cùng một file có thể xuất hiện ở cả train và validation. Do cửa sổ trượt có overlap 50%, nhiều mẫu liên kề gần như cùng biểu diễn một đoạn chuyển động. Điều này làm điểm số tăng giả tạo vì mô hình học đặc trưng riêng của phiên ghi thay vì học hành vi người dùng. Giải pháp là dùng *StratifiedGroupKFold* theo group `user__file`, đảm bảo cùng một file không bị tách sang hai tập.

6.3 Ít file ở một số người dùng

Số file inertial giữa các user không đều. User ít nhất là `userQ`, chỉ có 3 file inertial. Nếu cố dùng 5-fold, một số fold validation có thể thiếu user này, làm macro-F1 không đánh

giá đủ class và khiến kết quả kém tin cậy. Giải pháp là chọn 3-fold, cân bằng giữa số lần đánh giá và điều kiện mỗi fold vẫn có đủ 12 user known.

6.4 Mất cân bằng số cửa sổ giữa các file

Một số file dài sinh ra rất nhiều cửa sổ, khiến một vài người dùng hoặc phiên ghi chỉ phối dữ liệu huấn luyện. Giải pháp là giới hạn tối đa 300 cửa sổ mỗi file và lấy mẫu đều theo thời gian bằng `np.linspace`.

6.5 Dữ liệu cảm biến có nhiễu và khác tần số

Dữ liệu thực tế từ điện thoại có thể lệch tần số lấy mẫu hoặc nhiễu cao tần. Pipeline xử lý bằng cách resample về 50 Hz, low-pass 20 Hz và high-pass 0.3 Hz cho gia tốc.

6.6 CNN không vượt Random Forest

Ban đầu kỳ vọng CNN 1D sẽ tốt hơn do học trực tiếp từ chuỗi cảm biến. Tuy nhiên, kết quả simple cho thấy Random Forest đạt macro-F1 0.7845, cao hơn CNN 0.6881. Nhóm chọn báo cáo trung thực kết quả này và lý giải rằng với dữ liệu nhỏ, đặc trưng thủ công vẫn hiệu quả hơn mô hình học sâu.

6.7 Open-set còn yếu

AUROC open-set chỉ đạt khoảng 0.67 với softmax max-prob. Nguyên nhân là embedding chưa đủ ổn định khi số phiên/người còn ít. Giải pháp đề xuất là tăng số phiên ghi, thu thêm dữ liệu unknown và thử các phương pháp học biểu diễn như contrastive learning ở giai đoạn sau.

6.8 Tái lập thí nghiệm open-set

Khi rà soát lại thí nghiệm, nhóm nhận thấy phần open-set phải tự tách known/unknown trong `openset.py`. Nếu dùng lại helper của closed-set, script có nguy cơ chỉ đọc phần known và không đánh giá đúng các user unknown. Phiên bản hiện tại tạo mask theo `UNKNOWN_USERS`, đồng thời đọc được cả checkpoint cũ và mới. Nhờ vậy lệnh `python -m src.openset` có thể chạy lại để sinh `openset_results.json`.

7 Khám phá mới hoặc kết luận

7.1 Khám phá chính

1. **Random Forest là lựa chọn phù hợp nhất cho báo cáo chính.** Mô hình này đạt điểm cao hơn CNN, dễ giải thích hơn và ít tham số lạ với người mới học khoa học dữ liệu.

2. **Đánh giá group-by-file là bắt buộc.** Nếu không nhóm theo file, kết quả có thể bị leakage và không phản ánh khả năng tổng quát hoá tới phiên ghi mới.
3. **Dữ liệu chuyển động giàu thông tin hơn khi hoạt động có chuyển động rõ.** Các hoạt động như walking/standing thường cho tín hiệu phân biệt người dùng tốt hơn sitting.
4. **Deep learning không phải lúc nào cũng thắng.** Trong bối cảnh dữ liệu nhỏ, mô hình cổ điển với feature tốt có thể vượt mô hình CNN đơn giản.
5. **Open-set cần thêm dữ liệu và phương pháp chuyên biệt.** Softmax max-prob chỉ là baseline, chưa đủ tốt cho triển khai thực tế.

7.2 Kết luận

Đồ án đã hoàn thành pipeline từ dữ liệu thô tới đánh giá mô hình: đọc CSV, chuẩn hoá manifest, tiền xử lý cảm biến, chia cửa sổ, trích đặc trưng, huấn luyện Random Forest và CNN 1D, đánh giá closed-set/open-set và sinh biểu đồ báo cáo.

- Random Forest: mô hình chính, đạt macro-F1 0.7845 và accuracy 0.8013.
- CNN 1D: mô hình học sâu bổ sung, đạt macro-F1 0.6881.

7.3 Hướng phát triển

- Thu thêm dữ liệu nhiều phiên hơn cho mỗi người dùng.
- Kết hợp inertial với keystroke ở mức feature fusion.
- Cải thiện open-set bằng threshold calibration hoặc học biểu diễn tương phản.
- Tạo giao diện demo đơn giản để upload CSV và dự đoán người dùng.

7.4 Đóng gói sản phẩm nộp

- Mã nguồn dự án và cấu hình thí nghiệm.
- Hướng dẫn cài đặt và chạy: `README.md` và `requirements.txt`.
- Báo cáo cuối: `bao_cao.pdf`.
- Notebook minh hoạ: `simple_pipeline.ipynb`.
- Kết quả số: `simple_results.json`.
- Các hình kết quả dùng trong báo cáo.

Tài liệu

- [1] Pedregosa et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, 2011.
- [2] Paszke et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, NeurIPS, 2019.
- [3] Virtanen et al., *SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python*, Nature Methods, 2020.
- [4] Anguita et al., *A Public Domain Dataset for Human Activity Recognition Using Smartphones*, ESANN, 2013.